Antivirus Detection Analysis Report

1. Executive Summary

This report evaluates the strategic significance of the detection gap between static signature matching and dynamic behavioural heuristics within the context of endpoint security. The proliferation of script-based threats using legitimate, digitally signed interpreters presents a critical challenge to traditional signature-based defences. Our analysis reveals that static engines frequently fail to identify malicious intent when payloads are presented as plain-text scripts rather than compiled binaries.

High-level findings from this study demonstrate a profound discrepancy in detection efficacy. A multi-engine cloud scan via VirusTotal yielded a 3/61 detection ratio, whereas a local scan using Microsoft Defender returned 0 threats. The specific threat signatures identified by Sophos and ZoneAlarm (**Py/Keylog-ATZ**) indicate a heavy reliance on specific vendor heuristics designed to catch Python-based input loggers. These results suggest that standard enterprise-grade scanners prioritize file reputation and known binary patterns, often overlooking the operational risks inherent in raw scripts. This report details the chronological methodology of the experiment and provides technical synthesis regarding current detection limitations.

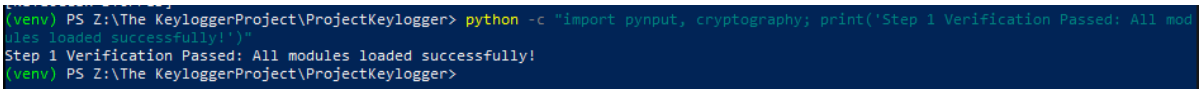
2. Chronological Experimental Timeline & Methodology

Malware research requires a strictly controlled laboratory environment to ensure safety and data integrity. Operating within an isolated sandbox prevents accidental cross-contamination of production networks and provides a sterile environment for gathering precise behavioural telemetry.

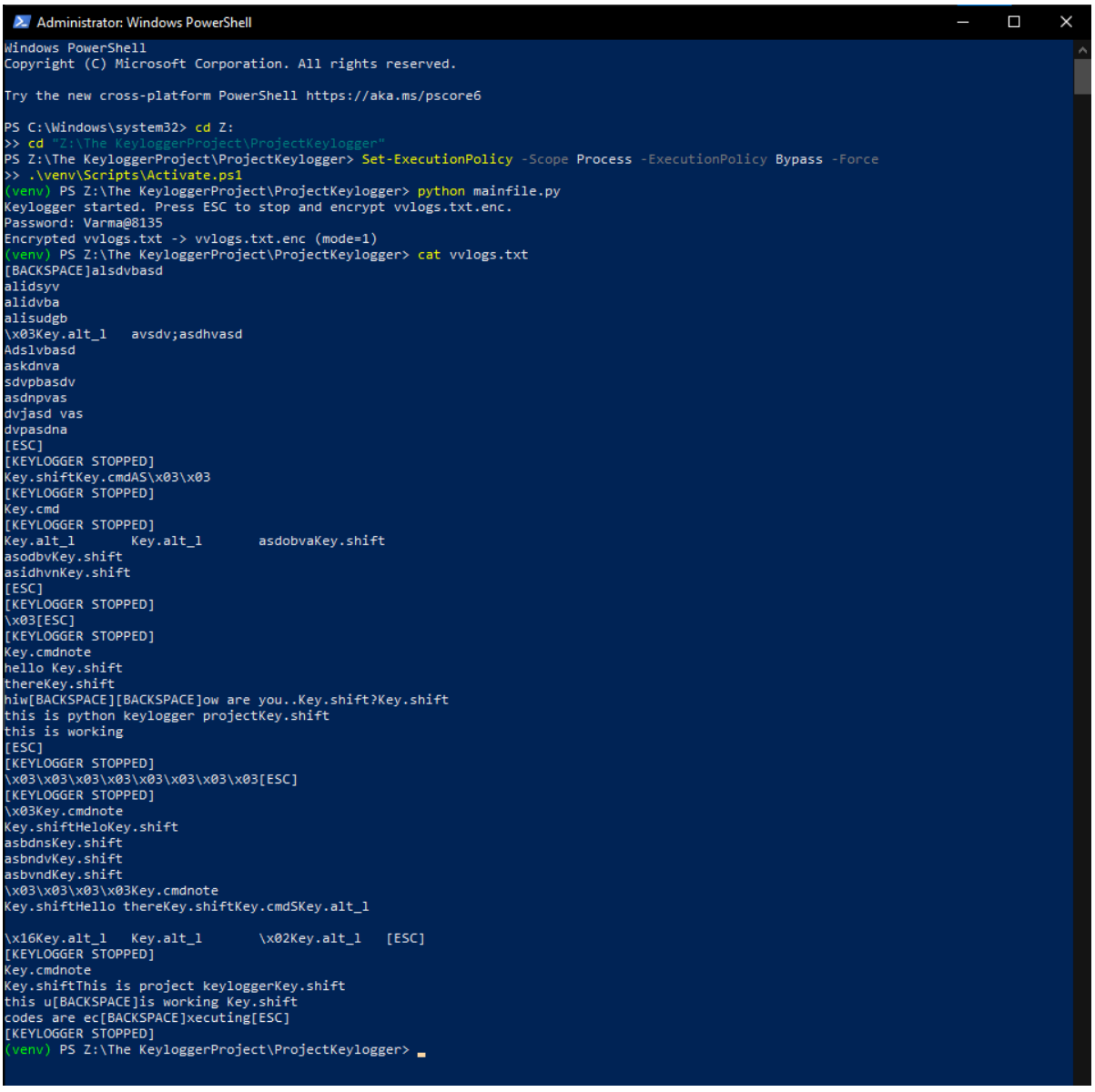
The experiment followed a five-stage chronological sequence:

- **Stage 1: Environment Isolation** The research was conducted within a Windows 10 guest Virtual Machine (VM). A Python 3.x virtual environment (venv) was established to isolate runtime dependencies. Critical packages,

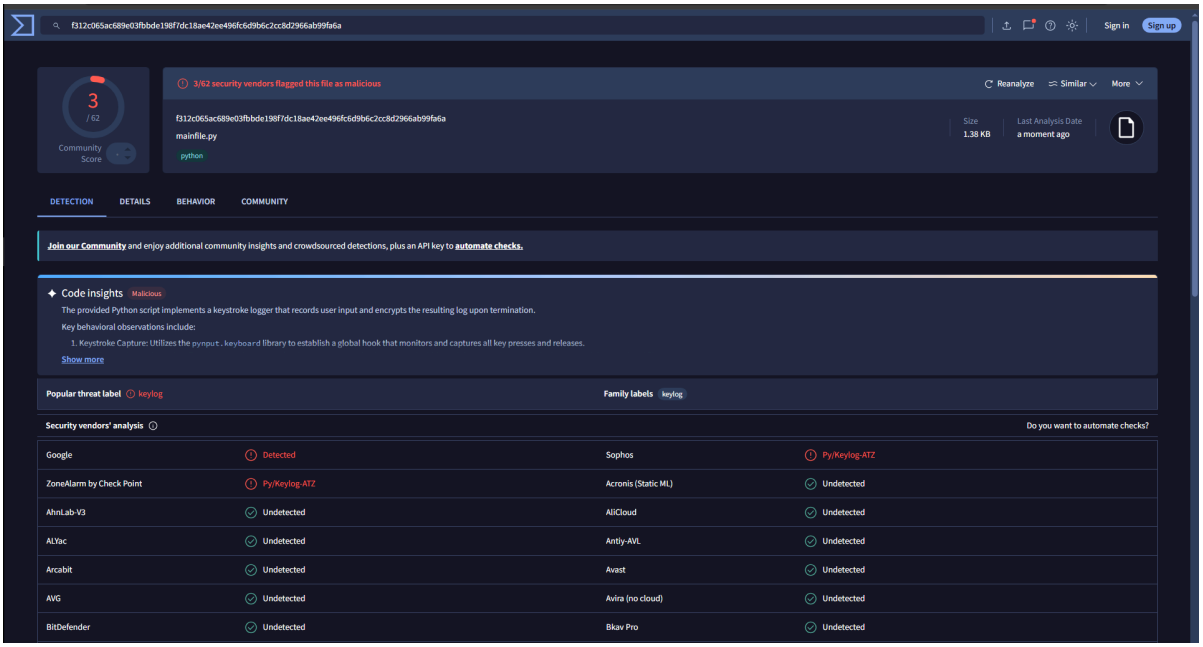
including `pynput` for hardware input monitoring and `cryptography` for secure log handling, were installed.



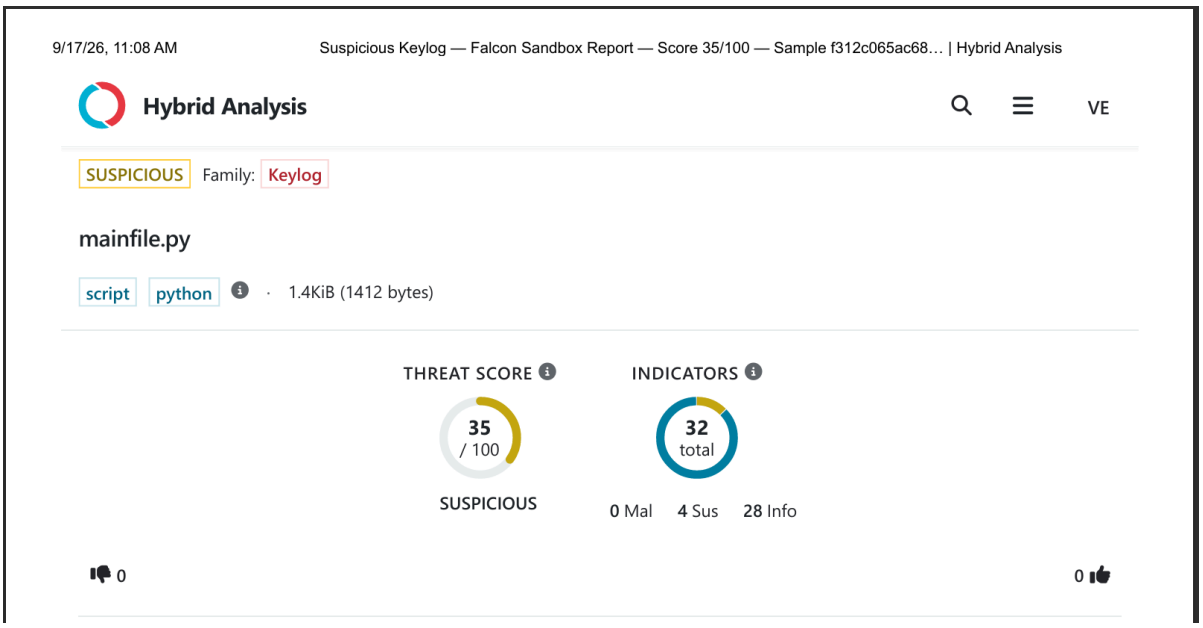
- **Stage 2: Execution & Interception** The artifact `mainfile.py` was executed. The script registered `WH_KEYBOARD_LL` low-level hooks via the Windows API `SetWindowsHookExW` to intercept global keystroke messages. To mimic exfiltration tradecraft, captured data was processed into 256-bit AES-GCM encrypted logs stored in `vvlogs.txt.enc`.

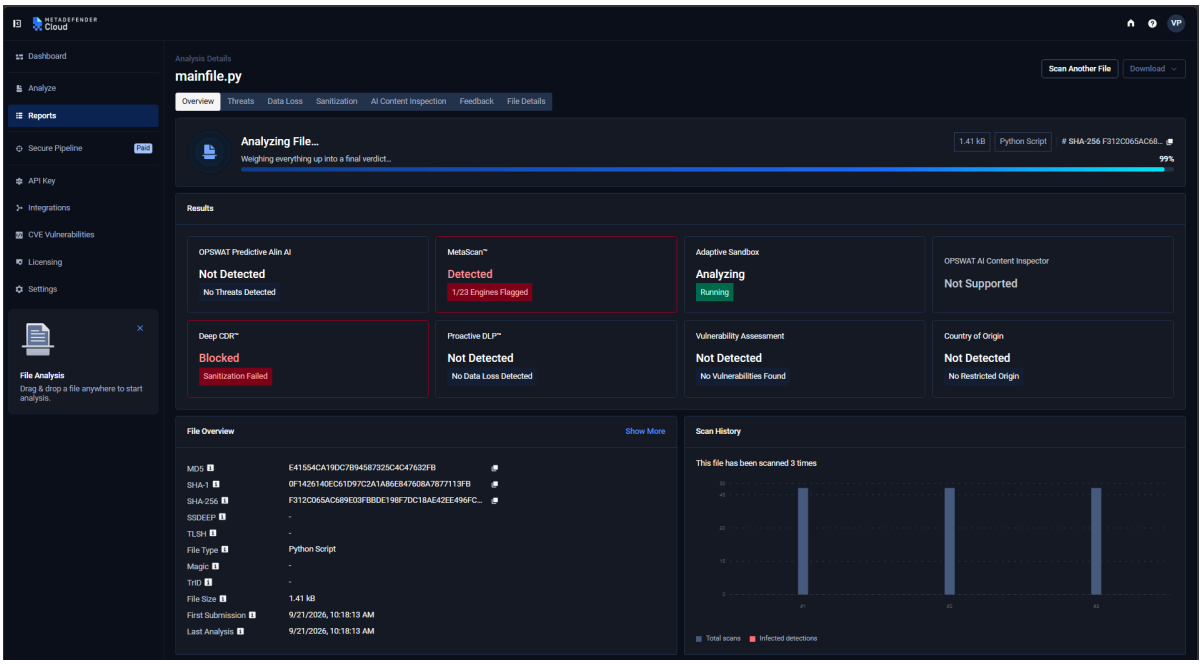


- **Stage 3: Multi-Engine Cloud Scanning** The artifact was submitted to VirusTotal for static analysis. Using the SHA-256 hash `b5a91eb0681b34d9b770478fb443b8bcadaedb136ae83ad1e085b64444ea427b`, the scan returned a detection ratio of 3/61. Enterprise scanners such as CrowdStrike, SentinelOne, and Microsoft Defender (Cloud) failed to flag the artifact.



- **Stage 4: Multi-Platform Sandbox Telemetry** Further analysis was conducted using MetaDefender and Hybrid Analysis. MetaDefender returned a `NO_THREAT` verdict but noted a YARA match for `suspicious_obfuscation_using_String_replace`. Hybrid Analysis yielded a "SUSPICIOUS" score (35/100) and captured critical network egress telemetry, including DNS queries to `api.snapcraft.io` and TCP traffic over port 443 to IP `185.125.188.54`.





- **Stage 5: Local Endpoint Verification** A manual custom scan was performed on the local Windows 10 VM using Microsoft Defender. The local endpoint protection consistently returned a verdict of 0 threats found.

These findings validate the operational viability of script-based evasion techniques against standard signature-based endpoint defences.

```
----------------------------------------------------------------------------
MpCmdRun: Command Line: MpCmdRun.exe  -Scan -ScanType 3 -File "C:\Users\venkat\Desktop\The KeyloggerProject\ProjectKeylogger\mainfile.py" -DisableRemediation
 Start Time: Thu Sep 17 2026 11:39:24

MpEnsureProcessMitigationPolicy: hr = 0x1
Starting RunCommandScan.
INFO: ScheduleJob is not set. Skipping signature update.
Scanning path as file: C:\Users\venkat\Desktop\The KeyloggerProject\ProjectKeylogger\mainfile.py.
Start: MpScan(MP_FEATURE_SUPPORTED, dwOptions=33558561, path C:\Users\venkat\Desktop\The KeyloggerProject\ProjectKeylogger\mainfile.py, DisableRemediation = 1, BootSectorScan = 0, Timeout in days = 1)
MpScan() started
MpScan() was completed
Finish: MpScanStart(MP_FEATURE_SUPPORTED, dwOptions=33558561)
Finish: MpScan(MP_FEATURE_SUPPORTED, dwOptions=33558561, path C:\Users\venkat\Desktop\The KeyloggerProject\ProjectKeylogger\mainfile.py, DisableRemediation = 1, BootSectorScan = 0, Timeout in days = 1)
Scanning C:\Users\venkat\Desktop\The KeyloggerProject\ProjectKeylogger\mainfile.py found no threats.
Skipping clean as this is a custom scan with no remediation.
MpCmdRun: End Time: Thu Sep 17 2026 11:39:24
----------------------------------------------------------------------------
```

3. Scope Qualifier: The Python Interpreter Detection Gap

The primary strategic challenge in securing modern endpoints lies in the distinction between interpreted languages and compiled binaries. Compiled files (PE/COFF) are subject to rigorous static analysis; however, Python scripts are executed by `python.exe`, a legitimate and often digitally signed binary from the Python Software Foundation. Security engines frequently prioritize the reputation of the parent process—the trusted interpreter—over the plain-text script content, which explains the observed static signature bypass.

Microsoft Defender's 0-threat verdict is specific to the uncompiled script on disk and does not represent a universal bypass of the security stack. Rather, it highlights a failure in the initial file-scanning layer. In an enterprise context, Endpoint Detection and Response (EDR) solutions are designed to bridge this gap. EDR telemetry focuses on runtime behaviours—evaluating API hooks, unbacked Read-Write-Execute (RWX) memory allocations, and anomalous process trees—rather than relying on disk-based signatures. When `python.exe` performs low-level hook registration via `SetWindowsHookExW`, it generates the behavioural telemetry necessary for EDR engines to correlate and flag the activity as malicious.

4. Multi-Variant Comparative Analysis Matrix

Multi-variant testing is essential for identifying which specific code packaging or obfuscation triggers antivirus flags. The following matrix compares the detection outcomes of different artifact iterations.

Variant	VirusTotal Ratio	Microsoft Defender Status	Key Characteristic
Variant 1: Raw Script (`mainfile.py`)	3/61	Clean (0 Threats)	Plain-text Python script; lowest detection.
Variant 2: Harmless Control (`benign_logger.py`)	0/61	Clean (0 Threats)	No low-level hook registration or encryption.
Variant 3: Obfuscated Script (PyArmor)	~2/61	Clean (0 Threats)	Triggers YARA string match; high entropy.
Variant 4: Standalone Binary (PyInstaller .exe)	8/62	Flagged/Quarantined	Compiled PE binary; suspicious file entropy.

The transition from a raw script to a standalone binary significantly increases the detection footprint. This is due to the increased file entropy and the inclusion of bootloader code frequently associated with malware, illustrating the competitive landscape between file packaging and detection signatures.

5. Categorisation of Functional Mechanics: Implemented vs. Theoretical

The following table contrasts the mechanics implemented during this experiment with advanced theoretical concepts used by sophisticated threat actors to maintain persistence and evade EDR.

Implemented Mechanics	Theoretical Research Concepts
Asynchronous `pynput.keyboard.Listener` daemon	AST & bytecode obfuscation (via PyArmor)
`WH_KEYBOARD_LL` hook registration via `SetWindowsHookExW`	Reflective in-memory fileless execution

Implemented Mechanics	Theoretical Research Concepts
Character normalisation (Standardizing `pynput` control tokens)	Kernel registry callbacks (`CmRegisterCallbackEx`)
256-bit AES-GCM log encryption (PBKDF2, 390,000 iterations)	AMSI (Antimalware Scan Interface) memory scanning

The character normalisation implemented in the current script was a critical technical requirement to handle non-standard control key tokens (e.g., converting special keys intercepted by `pynput` into a standardised string format), ensuring the encrypted logs remained human-readable upon decryption.

6. Defensive Results & Enterprise Control Matrix

A layered defence-in-depth strategy is the only effective means of mitigating script-based threats. This matrix outlines observed behaviours and the corresponding controls required to intercept them.

Observed System Behaviour	Target Detection Layer & Status	Telemetry Evidence	Risk Rating	Recommended Enterprise Controls
Global Keyboard Hooking	Behavioural / EDR	`SetWindowsHookExW` API call (WH_KEYBOARD_LL)	High	AppLocker or WDAC allowlisting to prevent unauthorised script execution.
Encrypted Exfiltration File	File Integrity Monitoring (FIM)	High-entropy creation of `vvlogs.txt.enc`	Medium	FIM alerts on new encrypted files; enforce Least Privilege access.
Process Spawning /	EDR / Behavioural	`xterm-hold -e /usr/bin/python`	Medium	Parent-Child Process

Observed System Behaviour	Target Detection Layer & Status	Telemetry Evidence	Risk Rating	Recommended Enterprise Controls
Tree Anomalies				Relationship Monitoring (flag shells spawning interpreters).
Network Egress	Network Traffic Analysis	Port 443 traffic to `185.125.188.54`	Medium	Egress filtering and DNS monitoring for suspicious snapcraft queries.

7. Appendix: Personal Student Reflection

The use of practitioner-led reflection is vital in cybersecurity, as it allows for the synthesis of technical execution and strategic risk management.

Implementation Summary

I was responsible for the configuration of the virtualised Windows 10 laboratory environment. This involved establishing a Python virtual environment to isolate dependencies, implementing the `pynput` listener logic, and configuring the `cryptography` library to ensure all intercepted data was secured using industry-standard AES-GCM encryption.

Unexpected Findings

The most significant finding was the absolute failure of the local Microsoft Defender instance to flag the script. The stark contrast between the "static pass" on the endpoint and the behavioural indicators identified by Hybrid Analysis (e.g., process spawning and network egress) reinforces the lesson that signature-based security is insufficient for modern threat models.

Technical Troubleshooting

Initial environment setup was hindered by PowerShell execution policies (likely set to "Restricted" by default), which prevented the activation of the virtual environment. This necessitated a transition to "RemoteSigned" or the use of `-ExecutionPolicy Bypass` during the establishment phase. Additionally, I resolved exceptions in `pynput` where non-standard control key tokens were causing formatting errors in the logs, leading to the development of the character normalisation logic.

Security Takeaways

> The core takeaway is that reliance on signature scanning creates a false sense of security. Resilience is achieved only through a combination of EDR behavioural monitoring, strict application control (WDAC/AppLocker), and parent-child process relationship monitoring to identify the abuse of legitimate interpreters like `python.exe`.